

SQLite-Carver-Pro v1.9.2

Manuel de Référence Technique & Spécifications Internes d'Ingénierie Forensique

 **Déchiffrement SEE & SQLCipher, Carving WAL Slack, Multi-Version & Anti-Forensics**

Documentation approfondie décrivant les mécanismes physiques de stockage, le déchiffrement transparent de bases SQLCipher et SEE (SQLite Encryption Extension), l'analyse différentielle de trames WAL chiffrées, la récupération du WAL Slack, la reconstitution multi-version et la détection d'anti-forensics PRAGMA secure_delete.

Auteur : Dam-FOR3K

Version : 1.9.2 Édition Professionnelle Forensique)

Architecture Cible : Moteur Autonome Air-Gapped (Windows x86_64, Linux, macOS)

Conformité Méthodologique : ISO/IEC 27037 (Collecte et Préservation de Preuves Numériques)

1. Anatomie Physique d'une Base SQLite & Zones d'Audit

Contrairement aux outils de gestion de bases de données traditionnels qui s'appuient sur l'API officielle SQLite (`sqlite3.dll`), **SQLite-Carver-Pro** opère directement au niveau binaire sur les pages physiques brutes du fichier. Cette approche permet de s'affranchir des limitations du moteur de requêtes qui ignore systématiquement les données désallouées, tronquées ou corrompues.

1.1 L'En-Tête de Base de Données (100 Octets)

Les 100 premiers octets du fichier constituent l'en-tête de base de données (présent uniquement sur la Page 1). Chaque champ fournit des métriques critiques pour l'analyse judiciaire :

Offset	Taille	Nom du Champ	Description & Rôle Forensique
0	16 B	Magic Header String	Chaîne ASCII obligatoire : <code>SQLite format 3\0</code> . Valide l'intégrité de signature.
16	2 B	Page Size	Taille physique des pages (512 à 65536 octets, puissance de 2). Conditionne tous les calculs d'offset.
18	1 B / 1 B	Write / Read Version	Version de format de fichier (1 = Rollback Journal conventionnel, 2 = Write-Ahead Log / WAL).
20	1 B	Page Reserved Space	Zone critique : Nombre d'octets réservés à la fin de chaque page (0 à 255). Ces octets sont ignorés par le moteur B-Tree standard. Ils hébergent les IV/HMAC en mode SQLCipher (48 octets) ou sont exploitables pour de la dissimulation stéganographique anti-forensique.
24	4 B	File Change Counter	Incrémenté à chaque transaction d'écriture. Permet de détecter les falsifications ou restaurations non autorisées.
28	4 B	Database Size in Pages	Nombre de pages valides. Permet d'identifier immédiatement le <i>Slack EOF</i> (données cachées en fin de fichier).
32	4 B	First Freelist Trunk Page	Pointeur vers la première page tronc de la liste des pages libérées (freelist).
36	4 B	Total Freelist Pages	Nombre total de pages libérées et réutilisables dans la base de données.
52	4 B	Largest Root Page / Vacuum	Mode Auto-Vacuum détecté par SQLite-Carver-Pro v1.9.2 <code>NONE</code> (0), <code>FULL</code> , ou <code>INCREMENTAL</code> .
56	4 B	Text Encoding	Encodage textuel par défaut (1 = UTF-8, 2 = UTF-16le, 3 = UTF-16be).

1.2 Typologie des Pages B-Tree

Chaque page de la base de données est structurée selon l'un des 4 types B-Tree définis par son premier octet :

- **Table Leaf Page (0x0D)** : Contient les données applicatives réelles (cellules de données) et le RowID (clé primaire entière). C'est la source principale d'extraction.
- **Table Interior Page (0x05)** : Arbre de routage des tables contenant les clés RowID et les pointeurs de pages filles vers d'autres pages B-Tree.

- **Index Leaf Page (0x0A)** : Contient les clés d'index et le RowID cible. Est également utilisée pour le stockage des tables `WITHOUT ROWID` où la totalité des colonnes réside dans l'index.
- **Index Interior Page (0x02)** : Arbre de navigation des index pointant vers les pages feuilles d'index.

1.3 Compartiments Physiques d'une Page & Slack Forensique

Chaque page feuille B-Tree se divise en 5 zones distinctes au niveau physique :

- **En-Tête B-Tree (8 ou 12 octets)** : Spécifie le type de page, le premier freeblock, le nombre de cellules et le début de la zone de contenu de cellules.
- **Tableau de Pointeurs de Cellules (Cell Pointers Array)** : Liste d'entiers 16 bits croissants pointant vers les cellules actives.
- **Cell Slack / Espace Non-Alloué** : Espace contigu entre la fin du tableau de pointeurs et le début de la première cellule active. Contient d'anciennes cellules déréférencées.
- **Freeblocks (Blocs Libres Chaînés)** : Fragments de cellules supprimées répertoriés dans une liste chaînée (pointeur suivant + taille).
- **Espace Réservé (Reserve Bytes)** : Zone en fin de page préservée lors du chiffrement ou de configurations spécifiques.

2. Moteur Cryptographique Unifié : SEE & SQLCipher

2.1 Architecture du Chiffrement SQLite (SEE vs SQLCipher)

Dans les environnements mobiles et industriels, deux technologies majeures de chiffrement de bases SQLite coexistent :

Caractéristique	SQLCipher (v3 / v4)	SQLite Encryption Extension (SEE)
Algorithme & Mode	AES-256-CBC (avec vecteur IV par page)	AES-256-OFB ou AES-128-OFB (Output Feedback)
Dérivation de Clé	PBKDF2-HMAC-SHA512 (256 000 itér.) ou SHA1 (64 000 itér.)	Dérivation directe SHA-256 / SHA-1 ou clé brute
Octets Réservés (Reserve)	48 octets (16B IV + 20/32B HMAC-SHA)	0 ou 16 octets (selon variante avec ou sans nonce)
Structure Page 1	Octets 0-15 écrasés par le sel cryptographique aléatoire	Octets 16-23 stockés en clair (géométrie de page préservée)
Chiffrement du WAL	Trames de page chiffrées en AES-CBC	Trames de page chiffrées en AES-OFB

2.2 Détection Forensique de SEE par Analyse Géométrique

L'extension officielle SQLite Encryption Extension (SEE) ne chiffre pas les octets 16 à 23 de la Page 1 afin de permettre au pager SQLite de déterminer la taille des pages avant d'initialiser le chiffreur. **SQLite-Carver-Pro v1.9.2** exploite cette particularité forensique :

- L'en-tête de 16 octets `SQLite format 3` est chiffré (entropie élevée sur les 16 premiers octets).
- Les octets 16-23 sont analysés : taille de page valide (puissance de 2 entre 512 et 65536), versions de fichier 1 ou 2, et fractions de charge utile standard (`64`, `32`, `32`).
- Cette signature géométrique permet de qualifier une base SEE avec une **confiance de 98%** avant même la saisie du mot de passe.

2.3 Déchiffrement Transparent des Trames WAL (.db-wal)

Lors de l'utilisation du mode WAL avec SQLCipher ou SEE, l'en-tête de 32 octets du fichier WAL ainsi que les en-têtes de 24 octets de chaque trame sont écrits en clair par le Pager SQLite. En revanche, le corps de chaque trame (les 4096 octets de données B-Tree) est chiffré avec la clé maître.

Dans la version **1.9.2** `WalDiffEngine` extrait automatiquement la clé brute (`raw_key`) dérivée lors de la phase d'ouverture et déchiffre à la volée chaque trame en mémoire via `decrypt_wal_frame_page()`. L'analyste dispose ainsi d'un carving complet des données du WAL et de ses mutations sans aucune manipulation manuelle.

3. Innovations Forensiques Avancées (Deep Dive Bas-Niveau)

3.1 Carving du WAL Slack Space (Trames Obsolètes)

Lorsqu'un checkpoint WAL intervient (réécriture des pages du WAL dans la base principale), le fichier WAL n'est pas systématiquement tronqué. Il est souvent réinitialisé logiquement en incrémentant le numéro de sel (`salt1`). Les trames situées au-delà du dernier commit actif et conservant d'anciens numéros de sel constituent le **WAL Slack Space**.

Ces trames obsolètes renferment des versions antérieures de pages modifiées ou supprimées. **SQLite-Carver-Pro v1.9.2** détecte la rupture de sel dans la séquence transactionnelle et extrait automatiquement l'ensemble des cellules résiduelles avec la mention de provenance `wal_slack`.

3.2 Reconstitution Historique Multi-Version de Pages

Au sein d'un même fichier WAL ou entre plusieurs cycles de checkpoints, une même page physique peut être modifiée plusieurs fois. Le moteur forensique isole chaque version de page et construit un arbre historique des mutations, permettant de retracer l'évolution temporelle précise d'une ligne de données (valeur initiale, altérations successives, suppression finale).

3.3 Détection d'Anti-Forensics : PRAGMA secure_delete

Lorsqu'une base de données SQLite est configurée avec `PRAGMA secure_delete = ON` ou `FAST`, le moteur écrase immédiatement le contenu des cellules supprimées avec des octets nuls (`0x00`) avant de les chaîner en freeblock.

Le module d'intégrité de la **v1.9.2** analyse statistiquement le ratio de zéros au sein des freeblocks découverts. Un ratio supérieur à **90% de zéros** déclenche l'alerte d'intégrité `PRAGMA_SECURE_DELETE_DETECTED`, signalant à l'enquêteur une tentative délibérée ou native de destruction des artefacts résiduels.

4. Guide des Commandes & Utilisation Professionnelle

4.1 Interface Graphique Moderne (Desktop GUI)

```
# Lancement direct du GUI forensique moderne
sqlite-carver gui

# Ouverture immédiate d'une base chiffrée (SQLCipher ou SEE)
sqlite-carver gui evidence.db --key "MotDePasseOuCleHex64"
```

Le champ de saisie unifié **Decryption Key (SQLCipher / SEE)** de l'interface graphique accepte indifféremment une passphrase textuelle ou une clé hexadécimale brute (32 caractères pour AES-128 ou 64 caractères pour AES-256).

4.2 Ligne de Commande Forensique (CLI)

```
# 1. Carving complet d'une base avec déchiffrement transparent et analyse WAL
sqlite-carver carve evidence.db --key "cle_secrete" --export rapport.html

# 2. Carving spécifique du WAL Slack (trames obsolètes post-checkpoint)
sqlite-carver wal-diff evidence.db evidence.db-wal --slack --key "cle_secrete"

# 3. Chasseur de pages brutes dans un dump RAM sans en-tête SQLite
sqlite-carver carve-raw memory_dump.raw --workers 8 --export ram_carved.sqlite

# 4. Recherche forensique récursive de mots-clés dans les cellules, blobs et WAL
sqlite-carver search evidence.db "mot_cle_recherche" --key "cle_secrete"
```

Exécutable Autonome Certifié Air-Gapped

L'outil est compilé en binaire unique 64 bits autonome (`sqlite-carver-v1.9.2.exe`) fonctionnant directement sur les stations d'analyse forensique scellées sans aucune dépendance logicielle.